

The two orchestrators, explained with stick figures

Phase 2b — what we built, what we measured, and where the proof is

Team 23UG005 · Open5GS stack, 100 UEs · 8 October 2026

eMBB road — big, can wait



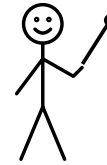
URLLC lane — small, must be fast



mMTC path — many tiny sensor reports ● ● ● ● ● ● ● ● ● ● ● ● ●



Slice orchestrator
= the traffic cop:
who goes on which
road, and who first

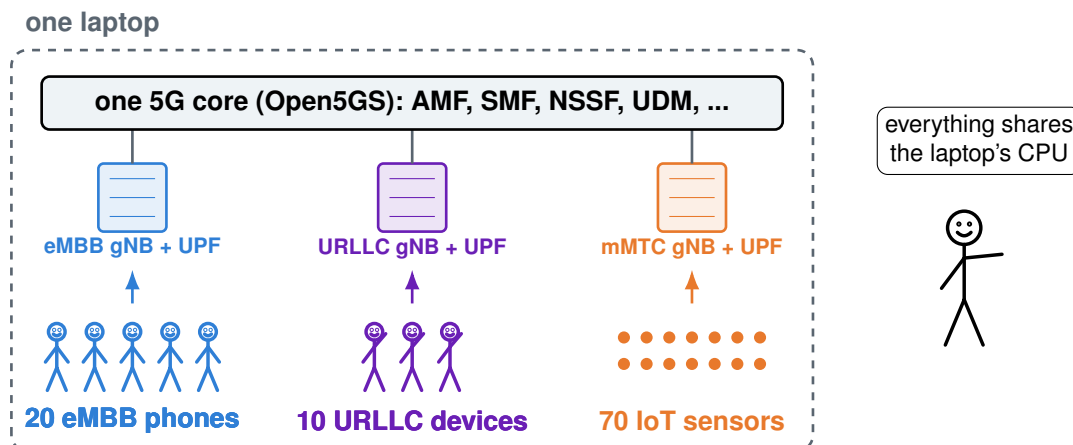


Network orchestrator
= the road builder:
opens and closes
whole roads

In one paragraph. Our 5G network has three slices, like three roads. The **slice orchestrator** is the traffic cop: it decides which road each request uses, lets VIPs (critical devices) go first, and slows down the big road when it starts hurting the fast lane. The **network orchestrator** is the road builder: it opens a whole road when someone needs it, closes it when nobody does, checks every step, and undoes its work if something goes wrong. Everything below was measured on our running 100-UE stack, and every number has a file behind it.

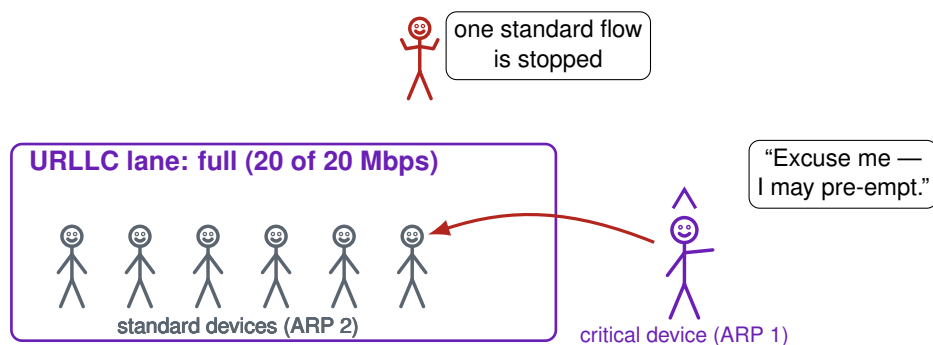
Contents 1 The setup 2 Traffic cop: priority 3 Traffic cop: the feedback controller 4 Traffic cop: planning 5 Road builder: off and on 6 Road builder: rollback 7 Road builder: on its own (closed loop) 8 Road builder: compute 9 The photos: one live run 10 Things that went wrong 11 Where every proof lives

1 The setup: 100 phones and sensors, three slices



Each slice has its own radio (gNB), its own user-plane (UPF) and its own data network. What they *share* is the laptop's CPU. Remember that — it turns out to matter in section 3.

2 Traffic cop: priority inside URLLC (ARP)

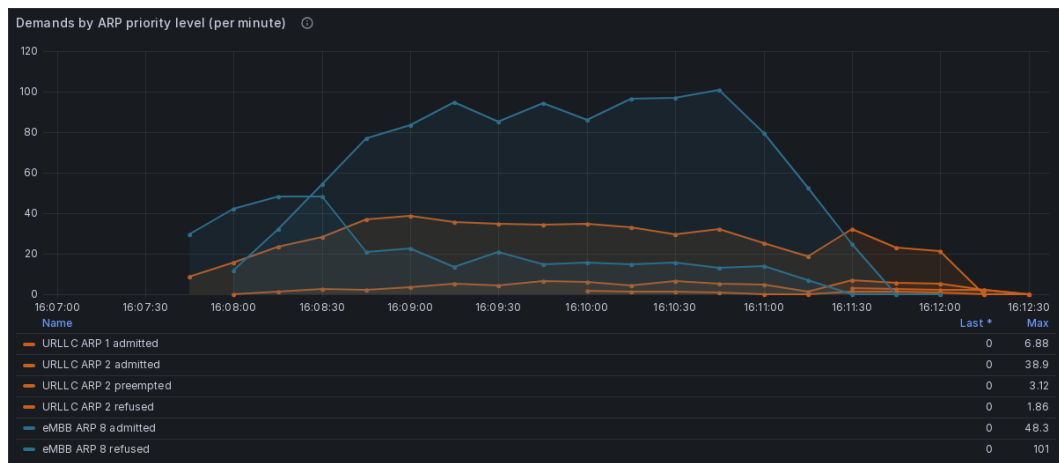


3GPP has a rule for exactly this: **ARP** (Allocation and Retention Priority). We gave 3 of the 10 URLLC devices ARP 1 ("critical, may pre-empt") and 7 of them ARP 2 ("standard, may be pre-empted"). The rule is stored in each device's subscription in the core.

What we did	What happened
20 standard demands	all admitted — the lane is full (20/20 Mbps)
1 more standard demand	refused : an equal priority cannot push anyone out
5 critical demands	all admitted , each by stopping exactly one standard flow
the flows afterwards	critical 5/5 got their full rate; the 15 standard flows left also got theirs

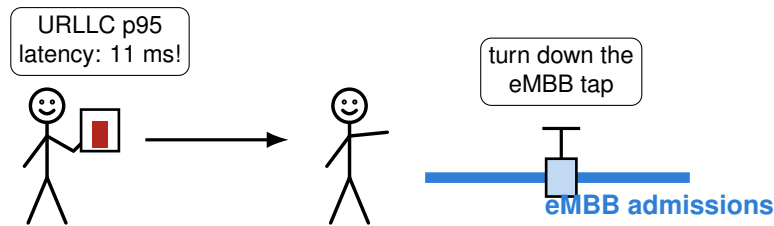
We ran it twice and got the same result both times.

Proof: docs/evidence/open5gs-orchestrator-dynamic/1-priority.txt



Grafana, from the live run in section 9: demands per minute by ARP level. “URLLC ARP 1 admitted” is the critical devices;
“URLLC ARP 2 preempted” is the standard flows they pushed out.

3 Traffic cop: the feedback controller



Every 5 seconds the controller looks at URLLC’s latency (the 95th percentile over the last 10 s). If it goes above 8 ms (80% of the 10 ms budget), it **cuts** how much eMBB traffic is admitted. When things are calm again and eMBB wants more, it **raises** the limit a little. That is the same “cut fast, grow slowly” rule TCP uses (AIMD).

Why eMBB? We measured it. Each kind of traffic alone, controller off:

Load (120 s)	URLLC p95	URLLC p99	host load
nothing (idle)	2.5 ms	5.4 ms	2.3
URLLC demand only	2.8 ms	5.3 ms	5.0
eMBB demand only	8.9 ms	17.5 ms	36.9

Proof: docs/evidence/open5gs-orchestrator-dynamic/4-diagnosis.txt

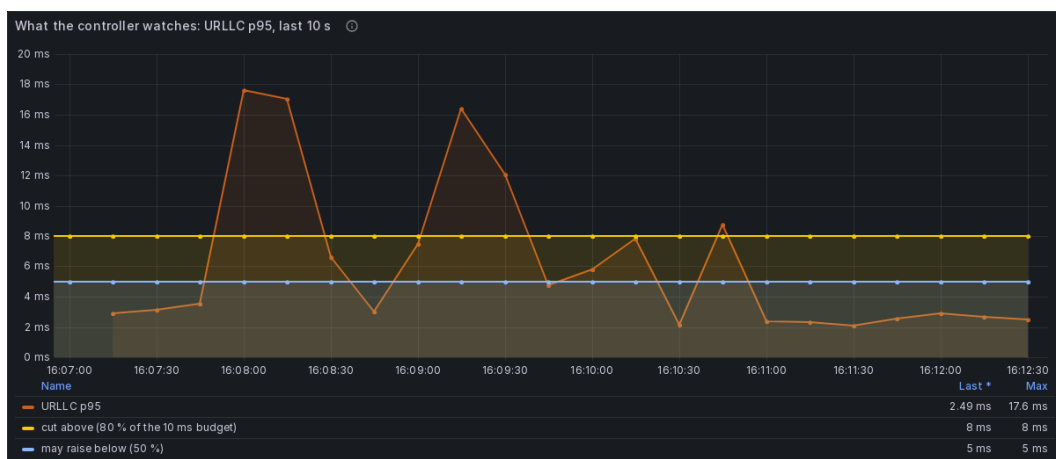
URLLC’s own traffic does not slow URLLC down. eMBB’s does — through the CPU they share.

The result — same demand, controller off vs on (180 s each):

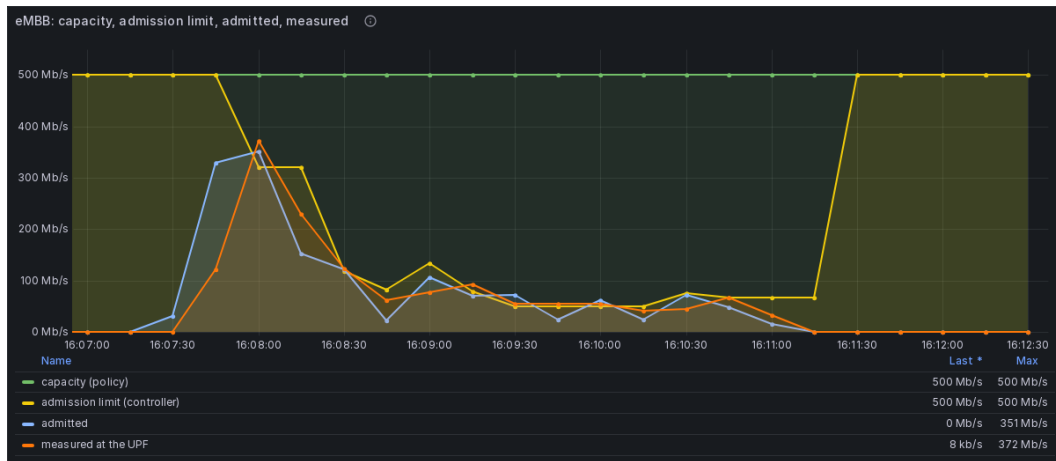
	controller off	controller on
eMBB flows that got their full rate	68 %	100 %
URLLC latency mean / p95 / p99	5.1 / 20.5 / 55.4 ms	3.2 / 12.5 / 34.0 ms
URLLC flows that got their full rate	100 %	100 %
eMBB data actually delivered	266 Mbps	109 Mbps

Proof: docs/evidence/open5gs-orchestrator-dynamic/2-comparison.txt

Honest note. The controller makes every admitted eMBB flow work and cuts URLLC’s slow tail by about 40 %, but it does *not* keep URLLC under the 10 ms target the whole time on this laptop, and it costs eMBB volume. Much of that “266 Mbps” was broken flows, though.

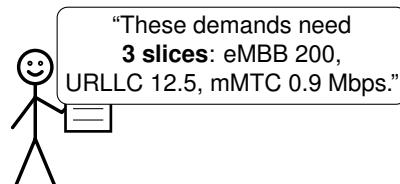


What the controller watches (live run, section 9): URLLC p95 over the last 10 s. Each time it climbs over the yellow 8 ms line, the controller cuts eMBB, and it comes back down.



eMBB in the same run: the policy capacity (green, 500 Mbps) and the controller's admission limit (yellow) falling from 500 to 50–130 Mbps under load. The jump back to 500 at 16:11:20 is the orchestrator being restarted for the priority test, not the controller.

4 Traffic cop: planning — how many roads do we need?



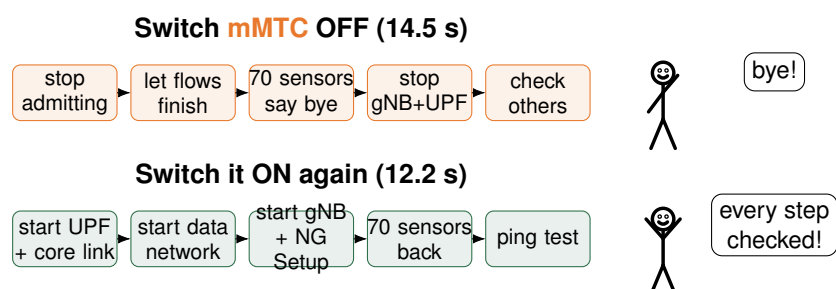
/plan looks at a set of demands (or what it actually saw in the last few minutes) and says which slices are needed and how much capacity each needs. Every kind of traffic goes on the cheapest road that is still good enough.

Demand	slices needed	capacity needed
Phase 2b mix: 10 control, 20 video, 70 sensors	3	eMBB 200, URLLC 12.5, mMTC 0.88 Mbps
only video and files (can wait)	1	eMBB 487.5 Mbps
what the orchestrator saw in a 180 s run	2	eMBB 363, URLLC 13.5 Mbps

Proof: docs/evidence/open5gs-orchestrator-dynamic/3-planning.txt

The road builder (next sections) is the one who *acts* on this answer.

5 Road builder: switching a slice off and on



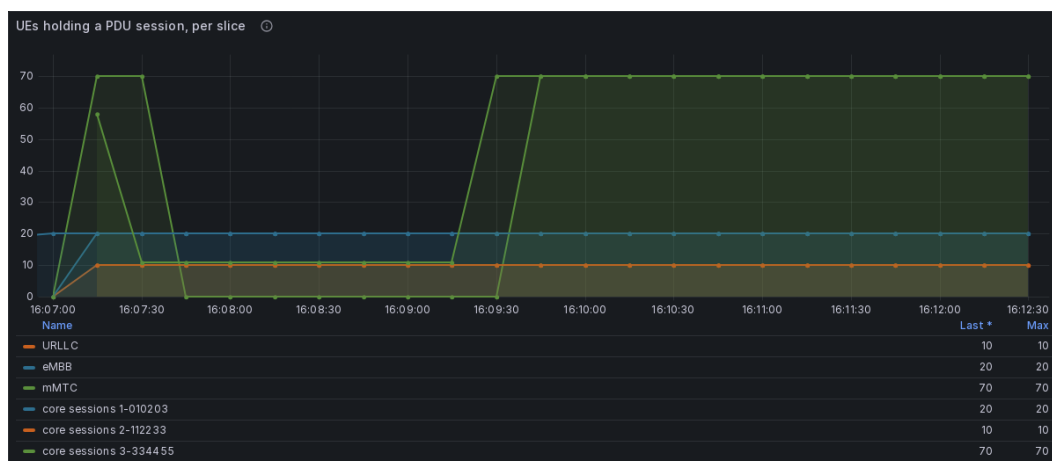
	mMTC on	mMTC off
mMTC sessions in the core	70	0
eMBB / URLLC devices still online	20 / 10	20 / 10
UE container CPU / memory	211 % / 200 MiB	40 % / 75 MiB
a sensor asks for service	admitted	refused: "mMTC is not active"

Proof: docs/evidence/open5gs-network-orchestrator/1-lifecycle.txt

While all this happened, URLLC's latency probes lost **0 of 63 891** packets (p95 2.8 ms): switching one slice did not disturb the others.



Grafana, live run (section 9): each slice serving (1) or dormant (0). mMTC goes off at 16:07:56 and back on at 16:09:30; eMBB and URLLC stay at 1 (their lines overlap).



Devices online per slice and the core's own session count. mMTC's devices go 70 → 0 → 70. In this run, under heavy load, the core still counted 11 mMTC sessions while the slice was off (see section 10).

6 Road builder: when something goes wrong, undo it



Proof: docs/evidence/open5gs-network-orchestrator/2-rollback.txt

A: no sensors **B: sensors knock** **C: sensors leave**

mMTC
switched OFF
after 67 s idle

road builder
sees the demand,
switches mMTC ON
(17 s)

OFF again
after 68 s idle

[illegible]

Proof: docs/evidence/open5gs-network-orchestrator/3-closed-loop.txt

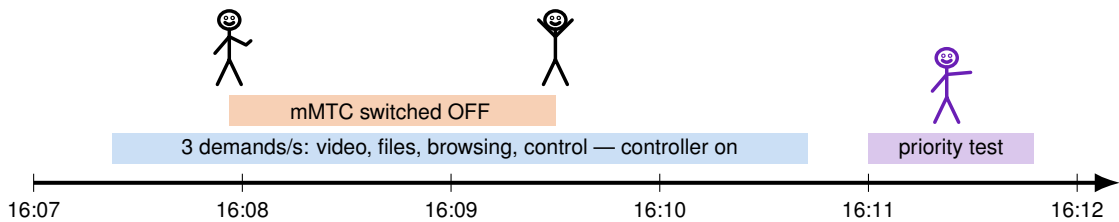
A stick figure on the left points towards a pie chart in the center. The pie chart is divided into three segments: a large blue segment (approximately 60%), a smaller orange segment (approximately 20%), and a smaller purple segment (approximately 20%). To the right of the pie chart is a speech bubble containing the text: "eMBB, you get only 1 CPU now."

Proof: docs/evidence/open5gs-network-orchestrator/4-compute.txt

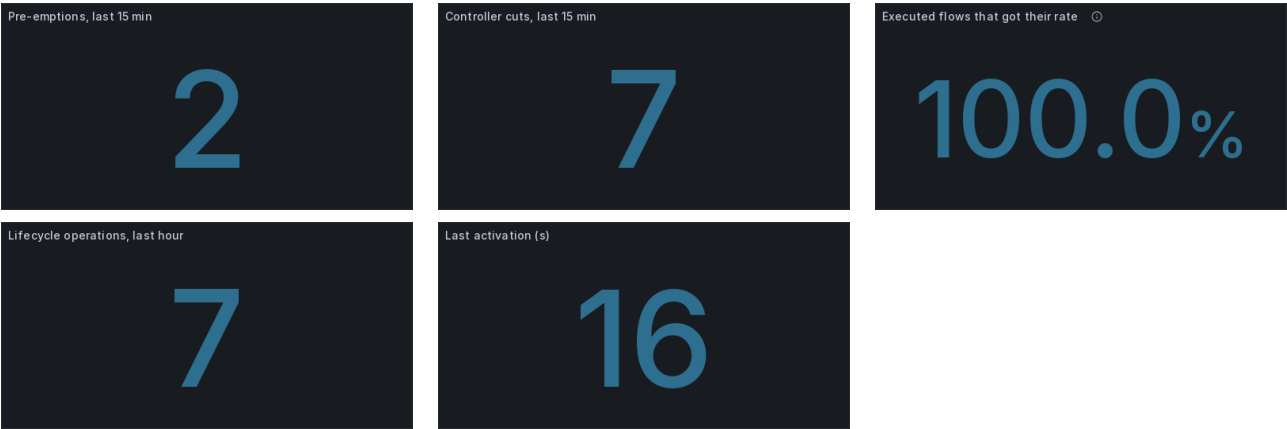
7

9 The photos: one live run, minute by minute

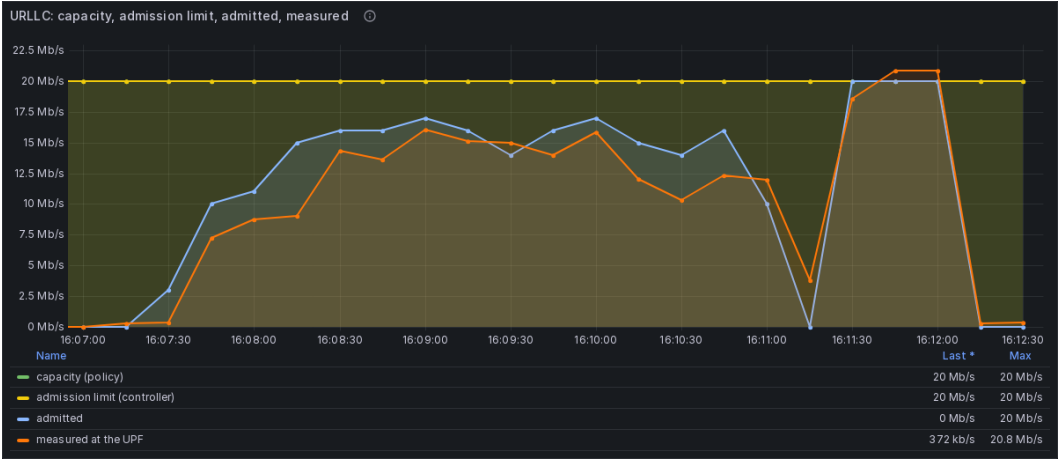
All the Grafana screenshots in this document come from **one scripted 5-minute run** on 8 October (16:07–16:12 UTC), so they show the same events. The numbers in the tables above come from the separate, longer evidence runs; this run is here so you can see it happen.



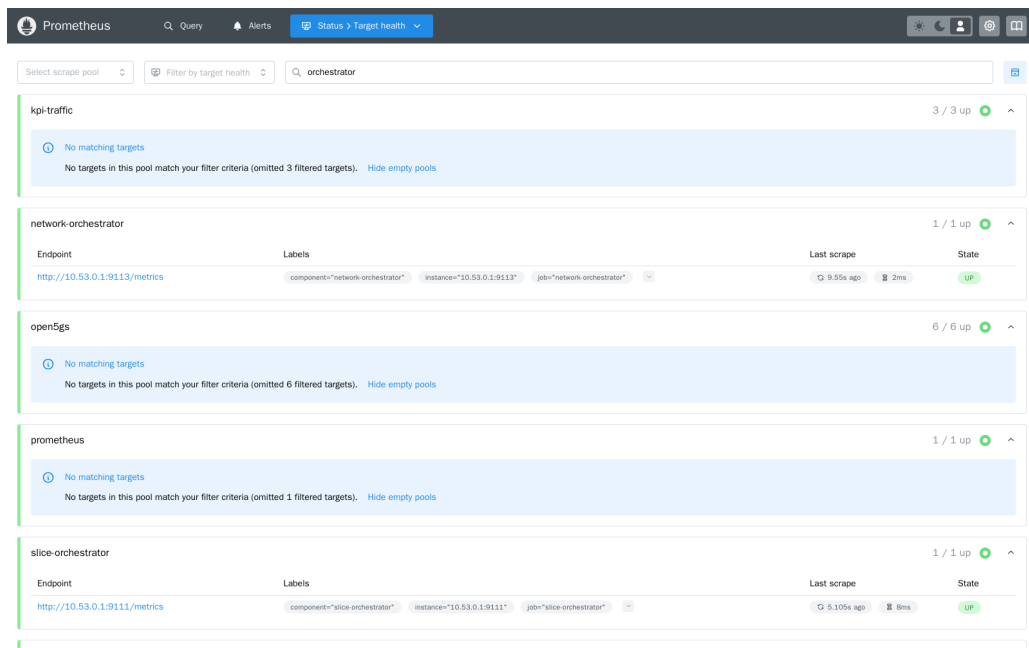
Time (UTC)	What happened
16:07:25	demand starts; the controller soon cuts eMBB's limit (500 → 320 → 120 Mbps) as URLLC p95 jumps to 17.6 ms
16:07:56	road builder switches mMTC off: done in 31.7 s (it waited for the busy host)
16:09:30	mMTC on again: done in 16.0 s
16:10:45	demand ends: 215 admitted, 265 refused (the controller and the dormant slice refused them)
16:11:20	priority test: 20 standard URLLC demands, then 5 critical ones



Counters from the same run: pre-emptions, controller cuts, executed flows that got their rate, lifecycle operations, and the last activation's time in seconds.



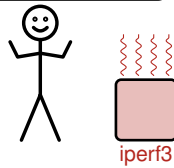
URLLC in the same run: what was admitted and measured on the fast lane (capacity 20 Mbps).



Prometheus scraping both orchestrators: `network-orchestrator (:9113)` and `slice-orchestrator (:9111)` are UP.

10 Things that went wrong (and what we learned)

"The thermometer
was heating the room."



Problem	What it did, and the fix
Our traffic tool was a CPU hog	iperf3's UDP sender used 45–90 % of a CPU core <i>per flow</i> , at any speed. The load it put on the laptop is what made URLLC slow, so the first two versions of the controller looked broken and even blamed the wrong slice. Fix: our own light flow tool (1–10 % of a core). The bad runs are kept, clearly marked as superseded.
Leaked processes	A timed-out flow left its sender running for 5 minutes at 90 % CPU. Fix: every flow runs under a hard kill timer.
Port clash	Flows were given the same port as our KPI collector, so 7 of 26 failed instantly. Fix: flows use ports from 5410.
Lost commands to devices	The UE command tool sent "deregister" to stale entries of old processes, so 38 of 70 never arrived. Fix: stale entries are cleaned by checking which ports are really in use.
Sessions left in the core	Devices were killed before their "I'm leaving" message went out. Fix: let each device switch itself off first. Now the core frees 69–70 of 70.
"Unlimited CPU" did nothing	<code>docker update -cpus 0</code> is silently ignored, but we reported success. Fix: unlimited = all 12 cores, and every change is read back.
Kubernetes would have broken	The UE pod would not have found its new helper script. Fixed in the generated manifests, not yet run on minikube.
Stale sessions under heavy load	In the live run (section 9), with the host busy, 11 of 70 sensors stopped before their "leaving" message reached the core, so the core kept counting them while mMTC was off. They clear when the sensors come back. Not fixed: switching off now reports it as a clear WARNING instead of a quiet detail.

The lesson for all of us: before trusting a measurement, check what the measuring tool itself costs. Every run now records the host's load and any leftover processes.

11 Where every proof lives

Folder / file	What is inside
docs/evidence/ open5gs-orchestrator-dynamic/	priority, controller on/off, planning, diagnosis — every flow and every controller decision as JSON
docs/evidence/ open5gs-network-orchestrator/	off/on, rollback, closed loop, compute — every operation with its step timings
docs/evidence/open5gs-orchestrator-dynamic/ superseded-iperf3-load/	the runs spoiled by iperf3 — kept on purpose, do not quote them
docs/screenshots/open5gs/10-*.png to 21-*.png	the Grafana screenshots in this document
docs/adr/ADR-016-dynamic-slice-allocation.md	the traffic cop's design decisions
docs/adr/ADR-017-network-orchestrator.md	the road builder's design decisions

To see it yourself (on the lab machine, stack running):

<code>make o5gs-network</code>	state of every slice
<code>python3 tools/network-orchestrator/netorch.py deactivate mMTC</code>	switch a slice off
<code>python3 tools/network-orchestrator/netorch.py activate mMTC</code>	...and on again
<code>make o5gs-dynamic</code>	re-run the traffic cop experiments (≈ 25 min)
<code>make o5gs-network-demo</code>	re-run the road builder experiments (≈ 16 min)
<code>http://localhost:3001</code> (Grafana)	rows "Dynamic allocation" and "Network orchestrator"

Still provisional — to confirm with our guide: 3 critical / 7 standard URLLC devices; the controller's 8 ms / 5 ms lines; mMTC as the only on-demand slice; 120 s idle time; 150 s activation deadline.

Phase 2b orchestrators: done, measured, and proven.

